



DEPLOYMENT PROCESS OF MICROSERVICES

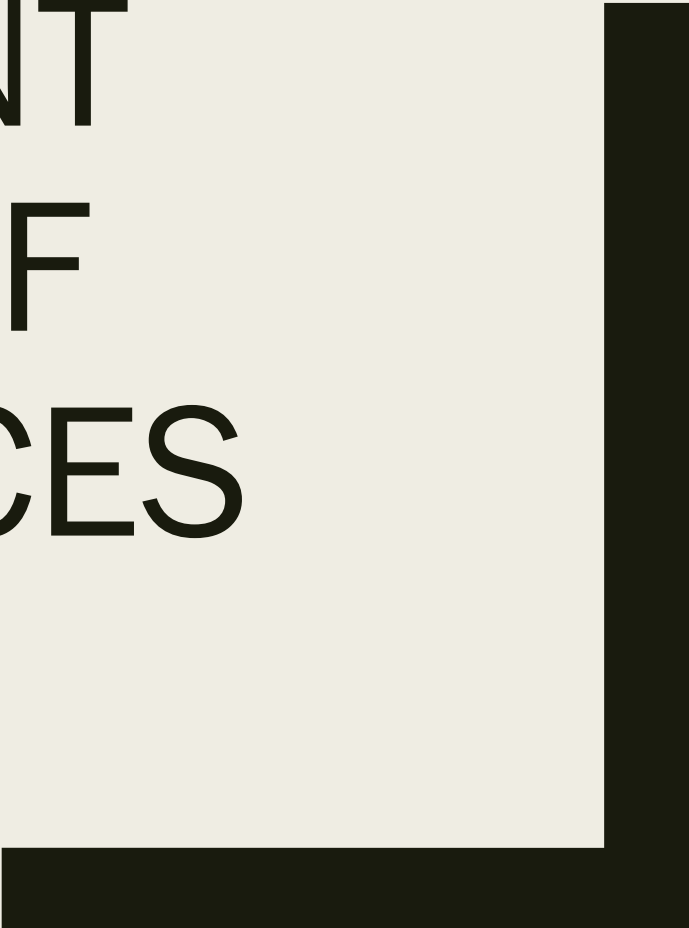
Faculty of Technology - University of Sri Jayewardenepura

Hansini Gunasena

Visiting Lecturer

Senior Software Engineer – Virtusa Pvt Ltd

BSc In Computer science



What is Microservice Deployment?

- Microservice deployment is the process of **making a single microservice ready to run and serve client requests** in an environment such as local, test, or production.
- Unlike monolithic systems, **each microservice is deployed separately**, without affecting other services.

Key Activities in Microservice Deployment

1. Building the Service

- Convert source code into a runnable artifact
- Examples:
 - JAR file (Spring Boot)
 - Docker image
- Includes:
 - Compiling code
 - Resolving dependencies
 - Running basic tests
- ✓ Ensures the service is technically ready to run

2. Configuring the Environment

- Set environment-specific values such as:
 - Server port
 - Database credentials
 - External service URLs
- Configurations change per environment:
 - Local
 - Development
 - Production
- ✓ Allows the same code to run in different environments

3. Running the Service

- Start the microservice process
 - The service:
 - Listens on a network port
 - Exposes REST APIs
 - Connects to its own database
- Example:
User Service →
`http://localhost:8081`
- ✓ Service becomes accessible to users or other services

4. Monitoring and Maintaining

- Track service health and performance
 - Detect failures and errors
 - Perform updates without downtime
- Examples:
- Log monitoring
 - Health checks
 - Restarting failed services
- ✓ Keeps the system reliable over time

Characteristics of Microservice Deployment

Independently Deployable

- Each service can be deployed **without stopping others**
- Enables faster development and bug fixing
- Reduces system-wide failures
- Ex : Updating Order Service does NOT affect User Service

Independently Maintainable

- Teams can own and maintain specific services
- Easier debugging and upgrades
- Smaller codebase per service
- Ex: One team manages Inventory Service only

Why This Matters

- Faster deployments
- Higher system reliability
- Better fault isolation
- Supports DevOps & CI/CD culture

Microservice deployment focuses on running, managing, and scaling each service independently, enabling faster development and easier maintenance compared to monolithic systems.

Typical Microservices Architecture

A microservices architecture is a system design where an application is built as a **collection of small, independent services**, each responsible for a **single business capability**.

Each service:

- Is developed and deployed independently
- Has its own database
- Communicates with other services over the network

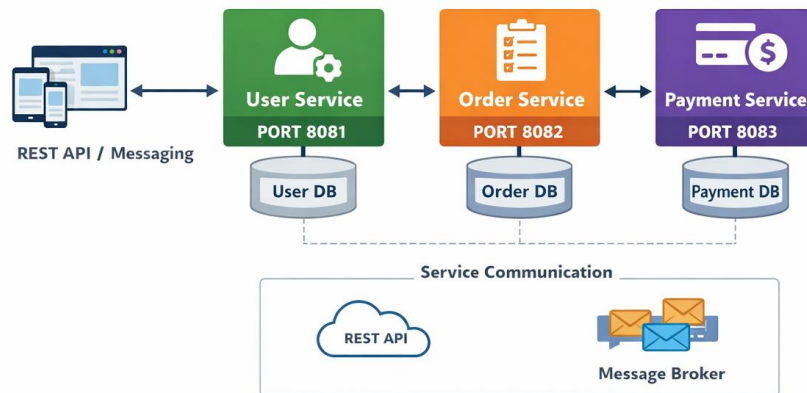
Components:

- Multiple microservices (User, Order, Payment, etc.)
- Each service runs on its own port
- Database per service
- Communication via REST / Messaging

Example:

- User Service → Port 8081
- Order Service → Port 8082
- Payment Service → Port 8083

Typical Microservices Architecture



Advantages of This Architecture

- Loose coupling
- Easy scaling
- Fault isolation
- Faster development cycles
- Technology flexibility

Local Deployment of Microservices

Local Deployment refers to running microservices on a **developer's own machine** instead of a cloud or production environment. It allows developers and students to **build, test, and understand microservices** before deploying them to real servers.

Used for:

- **Development** – writing and improving code
- **Testing** – verifying functionality and APIs
- **Debugging** – finding and fixing errors
- **Learning** – understanding microservices architecture and behavior

Tools:

- **IDE** – IntelliJ, Eclipse, VS Code (for coding & debugging)
- **Programming Languages** – Java, Node.js, Python
- **Docker (Optional)** – to run services in containers and simulate real environments

- Local deployment is the first step in the microservices lifecycle, enabling developers to safely develop and test services on their own machines.

- Steps in Local Deployment

1. Clone the microservice repository
2. Configure application properties
3. Start dependent services (DB, Config Server)
4. Run the microservice
5. Verify using browser or Postman

- Each service is started **independently**.

、

Slide 7 – Example: Local Deployment (Spring Boot)

Steps:

- Open project in IDE
- Configure application.yml
- Set server port
- Run using:
 - IDE Run button
 - mvn spring-boot:run

Service runs at:

<http://localhost:8081>

Configuration Management in Local Deployment

Common configurations:

- Server port
- Database URL
- Service endpoints
- Environment variables

Example:

```
yaml

server:
  port: 8081
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/userdb
```

■ Running Multiple Microservices Locally

■ Challenges:

- Port conflicts
- High memory usage
- Dependency order

■ Solutions:

- Assign different ports
- Use Docker Compose
- Start services one by one

What is Debugging in Microservices?

Debugging = Identifying and fixing issues in services.

- Common issues:
 - Service not starting
 - API errors
 - Database connection failures
 - Inter-service communication issues

Debugging Techniques

1.Logs

- Console logs
- Log files

2.Breakpoints

- Debug mode in IDE

3.Postman / Curl

- Test APIs

4.Health checks

- /actuator/health

Logging in Microservices

- Logging helps:
 - Track request flow
 - Identify errors
 - Debug production issues
- Log levels:
 - INFO
 - DEBUG
 - WARN
 - ERROR

Debugging Inter-Service Communication

■ Common problems:

- Wrong service URL
- Service down
- Timeout issues

■ Solutions:

- Verify service ports
- Use API Gateway (optional)
- Enable detailed logs
- Test services independently

- **Maintenance of Microservices**
- **Maintenance** = Keeping services healthy after deployment.
- Includes:
 - Monitoring
 - Updating services
 - Bug fixing
 - Performance tuning
- Microservices require **continuous maintenance**.

Monitoring in Local Deployment

Basic monitoring tools:

- Spring Boot Actuator
- Logs
- Simple dashboards

Actuator endpoints:

- /health
- /metrics
- /info

Updating a Microservice

Steps:

1. Make code changes
2. Test locally
3. Restart only affected service
4. Verify functionality

Advantage:

- ✓ No need to stop entire system

Common Maintenance Challenges

- Version compatibility
- Configuration mismatches
- Database schema changes
- Increased service count

Solution:

- Versioning APIs
- Centralized configuration
- Proper documentation

Best Practices

- One service = one responsibility
- Use environment-based configs
- Proper logging & monitoring
- Automate build & run
- Document APIs

Industry Perspective

In real-world:

- Local → Dev → Test → Production
- CI/CD pipelines
- Docker & Kubernetes
- Centralized logging and monitoring

Local deployment is the **first and most important step**.